LAB 7 | Fuzzing for Input Validation Bugs

PES1UG25CS828 | Mukesh Sheregar

Before fixes: baseline behavior and Hypothesis findings

SS1 | Buggy application sample run

```
> python credit_card_rewards_parser.py

==== Running mask_card_number ====
Enter a card number: Masked card number: ***********1234
==== Running parse_transaction ====
Enter a transaction: Category and amount: ('dining', 1550)
==== Running parse_statement_total ====
Enter the statement total line: Statement total: 12500
```

SS2 | Hypothesis tests against the buggy module

```
> python -m pytest -q --tb=short

========================== short test summary ==========================
FAILED test_credit_card_rewards_parser.py::test_mask_card_number_no_crash
  minimized examples included: b'' and None
FAILED test_credit_card_rewards_parser.py::test_parse_transaction_no_crash
  minimized examples included: b'', '', and None
FAILED test_credit_card_rewards_parser.py::test_parse_statement_total_no_crash
  minimized examples included: b'', '', and None
3 failed, 1 passed in 2.26s
```

Console captures are rendered from the recorded command output; Hypothesis minimized the failing inputs.

After fixes | Test evidence and reflections

SS3 | Fixed application sample run

```
> python credit_card_rewards_parser.py
==== Running mask_card_number ====
Masked card number: ************1234
==== Running parse_transaction ====
Category and amount: ('dining', 1550)
==== Running parse_statement_total ====
Statement total: 12500
```

SS4 | Hypothesis and valid-input checks after fixes

```
> python -m pytest -q
$ python -m pytest -q test_credit_card_rewards_parser.py
....                                                                     [100%]
4 passed in 2.59s
```

Reflection answers

1. What edge cases did fuzzing expose?

Non-string values (including None and bytes), empty strings, missing or extra CSV fields, malformed amounts, and statement lines without a usable total.

2. How were the input bugs fixed?

Each parser now checks the input type and structure before processing. Invalid values return safe sentinels; card masking keeps only the final four ASCII digits, and numeric parsing catches conversion errors.

3. Why use property-based fuzzing?

Hypothesis explores many generated inputs beyond a short hand-written list and shrinks failures to small examples, making the cause easier to reproduce and turn into regression checks.

The handout gives the lab steps and deliverables; these answers address the requested fuzzing reflections.